

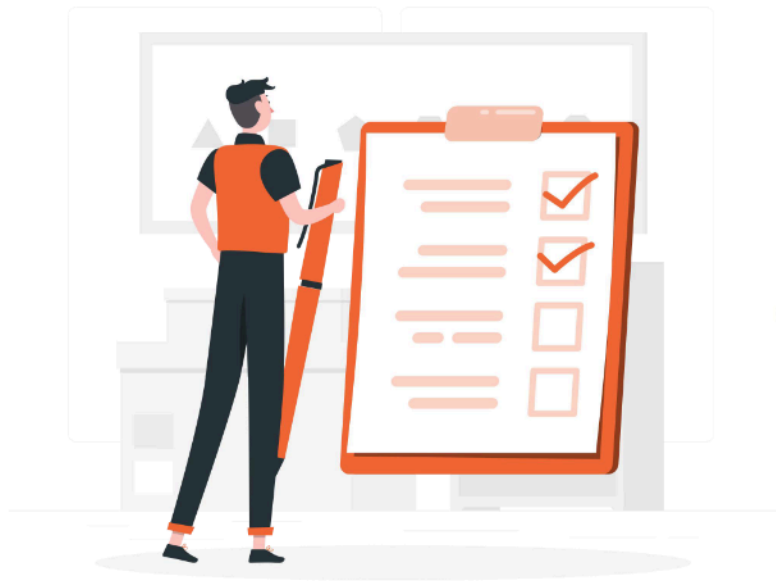
Build AI System with LangChain

Name | Job Title



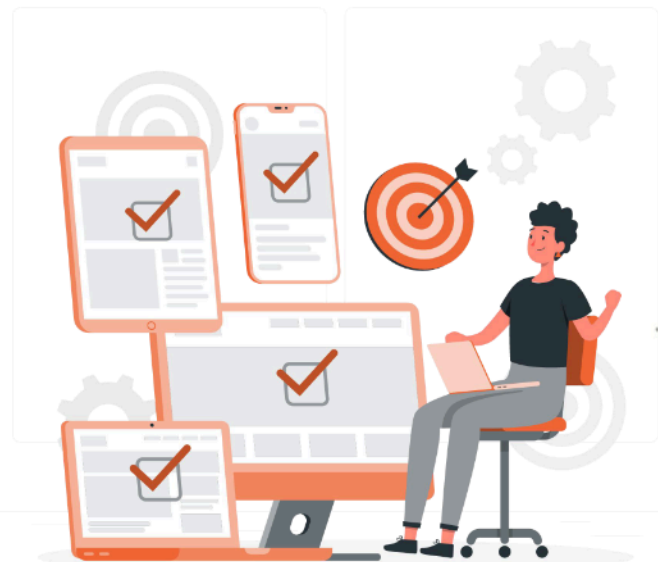
Today's Agenda

1. LangChain Core Building Blocks
2. LCEL (LangChain Expression Language)
3. Data Ingestion with Langchain
4. RAG Pipeline with Langchain



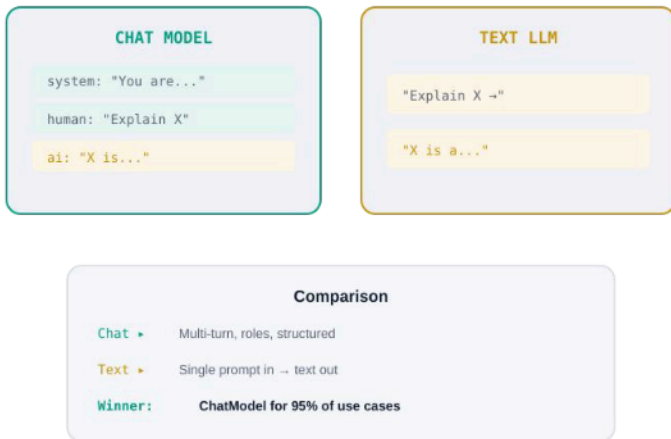
Learning Outcome

1. Understand LangChain Core building blocks
2. Understand LCEL Logic
3. Handling Data Ingestion with Langchain
4. Creating a modular RAG pipeline with LangChain





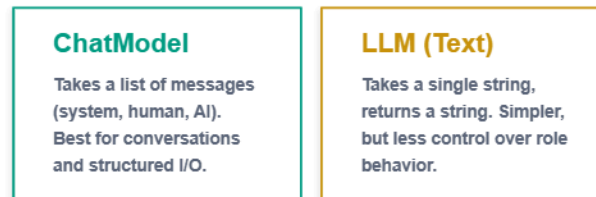
LangChain Core Building Blocks



LangChain Core

Model I/O

LangChain abstracts **two main model types**. Choosing the right one affects your entire chain.



Note: Use ChatOpenAI for almost every modern LangChain workflow.



LangChain



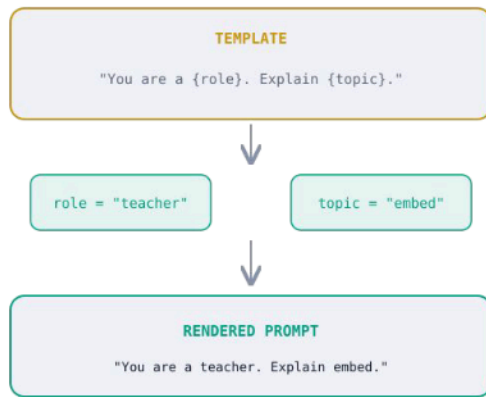
```
from langchain.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a {role}. "),
    ("human", "Explain {topic}.")
])

# Variables are injected at runtime
result = prompt.invoke({
    "role": "teacher", "topic": "embeddings"
})
```

LangChain Core Prompt Templates

Templates let you **parameterize your prompts with variables**, keeping logic separate from content.





LangChain



LangChain Core Output Parsers

LLMs return raw text. Parsers transform that text into structured data your code can use — like JSON, lists, or Pydantic objects.



Common parsers: JsonOutputParser, PydanticOutputParser, StrOutputParser, CommaSeparatedListOutputParser.

Basic Prompt + Model + Parser

```
from langchain_openai import ChatOpenAI
from langchain.prompts import ChatPromptTemplate
from langchain_core.output_parsers import JsonOutputParser

# 1. Define the Prompt Template
prompt = ChatPromptTemplate.from_messages([
    ("system", "Return JSON with keys: name, summary."),
    ("human", "Describe the concept: {concept}")
])

# 2. Initialize the Model
model = ChatOpenAI(model="gpt-4o-mini", temperature=0)

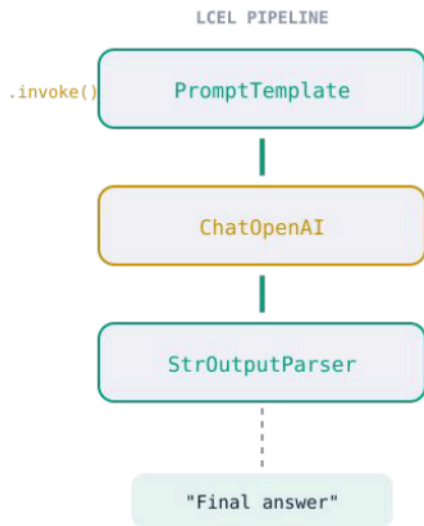
# 3. Choose an Output Parser
parser = JsonOutputParser()

# 4. Chain them together (preview of LCEL!)
chain = prompt | model | parser
result = chain.invoke({"concept": "vector embeddings"})
print(result)  # → {'name': '...', 'summary': '...'}
```

Combine all three core blocks into one executable script. The minimal building block of any LangChain application.



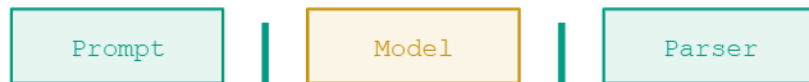
LCEL Logic



LCEL Logic

The Pipe Operator “|”

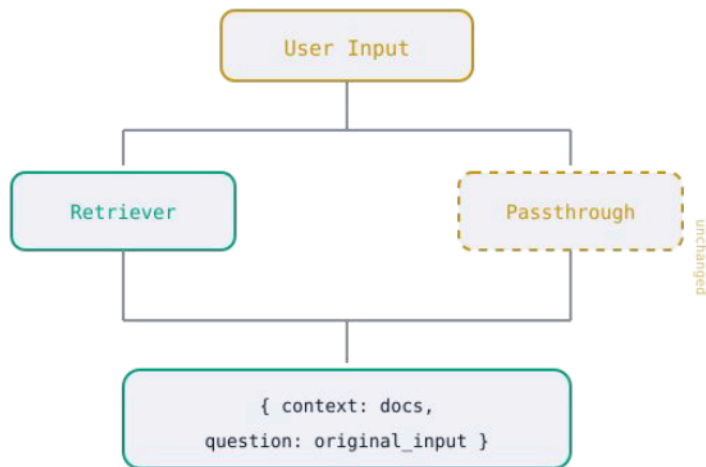
LCEL (LangChain Expression Language) uses the **pipe operator to connect components**.



Each component is a Runnable. The output of one feeds into the input of the next. This creates composable, streamable, batchable chains.



LangChain



LCEL Logic

RunnablePassthrough

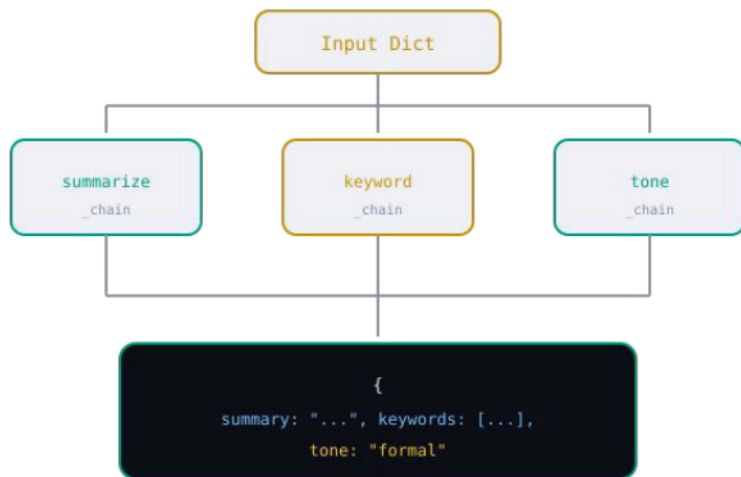
RunnablePassthrough forwards the **input data unchanged**, like a "pass-through wire" in a circuit. Use it when a later step needs the original input alongside data from earlier steps.

```
from langchain_core.runnables import RunnablePassthrough

chain = RunnableParallel(
    context = retriever,
    question = RunnablePassthrough()
) | prompt | model
```



LangChain



LCEL Logic

RunnableParallel

RunnableParallel runs **multiple chains simultaneously**. Each branch gets the same input, and all outputs merge into a single dict.



```

setup = RunnableParallel(
    summary = summarize_chain,
    keywords = keyword_chain,
    tone = tone_chain,
)
result = setup.invoke({"text": article})
  
```

Complete LCEL Chain



```
from langchain_openai import ChatOpenAI
from langchain.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableParallel

model = ChatOpenAI(model="gpt-4o-mini")

# Branch 1: Summarize
summarize = ChatPromptTemplate.from_template("Summarize:
{text}")
| model | StrOutputParser()

# Branch 2: Extract keywords
keywords = ChatPromptTemplate.from_template("5 keywords:
{text}")
| model | StrOutputParser()

# Combine and generate report
chain = RunnableParallel(summary=summarize, keywords=keywords)
| report_prompt | model | StrOutputParser()
```

Uses RunnableParallel to build a multi-branch analysis chain, merging results into a single prompt.



Data Ingestion with LangChain



LangChain

PyPDFLoader

CSVLoader

WebBaseLoader

DirectoryLoader

```
List[Document]
```

```
page_content: "The quick brown..."
```

```
metadata: {"source": "doc.pdf", "page": 1}
```

Data Ingestion

Data Loaders

LangChain provides **160+ loaders** that normalize data from different sources into a unified Document object.

Each Document has two fields:
page_content (text) and metadata
(source, page number, etc.).



Why Chunk?

1. Fits embedding model token limits
2. Better semantic search granularity
3. More relevant context injection

Data Ingestion

The Art of Chunking

LLMs have context limits. Documents must be **split into smaller, semantically meaningful chunks** before they can be embedded and searched.

- **chunk_size**: max characters per chunk (e.g., 1000)
- **chunk_overlap**: shared chars between chunks (e.g., 200)



LangChain



```
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(
    model="text-embedding-3-small"
)
vector = embeddings.embed_query("Hello world")
# → [0.012, -0.034, 0.056, ...] (1536 dims)
```

Data Ingestion

Embeddings

Embeddings transform text into high-dimensional vectors — dense numerical arrays that capture semantic meaning. Similar texts have vectors that are close together.



LangChain



```
from langchain_chroma import Chroma

# Create & persist a Chroma vector store
vectorstore = Chroma.from_documents(
    chunks, embeddings,
    persist_directory="./chroma_db"
)

# Query it
results = vectorstore.similarity_search("LCEL", k=4)
```

Data Ingestion

Vector Stores

Vector stores are specialized databases optimized for similarity search over high-dimensional vectors. Store vectors + metadata, then query by finding nearest neighbors.

FAISS

Chroma

Pinecone

Weaviate

Qdrant

...

Loading → Splitting → Saving

```
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import
RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain_chroma import Chroma

# 1. LOAD - Read PDF into Document objects
loader = PyPDFLoader("report.pdf")
docs = loader.load()

# 2. SPLIT - Chunk documents with overlap
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=200)
chunks = splitter.split_documents(docs)

# 3. EMBED + STORE - Create Chroma vector store
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vectorstore = Chroma.from_documents(
    chunks, embeddings, persist_directory="./chroma_db")
```

The full ingestion pipeline: load a PDF, split it into chunks, embed them, and store in a vector database.



RAG Pipeline with LangChain



LangChain



```
# Convert vector store → retriever  
retriever = vectorstore.as_retriever(  
    search_type="similarity",  
    search_kwargs={"k": 4}  
)
```

RAG with LangChain

The Retriever

A Retriever is the search interface for your vector store. It takes a query string and returns the most relevant documents.

Notes: k=4 means "return the 4 most similar chunks." Tune this based on your context window budget.



LangChain



```
rag_prompt =  
ChatPromptTemplate.from_template(  
    """Answer based on this context:  
    {context}  
    Question: {question}  
    """  
)
```

RAG with LangChain Context Injection

The retrieved documents are injected directly into the prompt as context. The LLM then generates an answer grounded in those facts.

Notes: The {context} variable is filled by the retriever's output. The {question} is forwarded by RunnablePassthrough.

The Final RAG Chain

```
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableParallel,
RunnablePassthrough
from langchain_chroma import Chroma

# Load the persisted Chroma vector store
vectorstore = Chroma(
    persist_directory="./chroma_db",
    embedding_function=OpenAIEmbeddings()
)
retriever = vectorstore.as_retriever(search_kwargs={"k": 4})

# RAG prompt with context + question slots
prompt = ChatPromptTemplate.from_template(
    "Answer using ONLY this context:\n{context}\n\nQuestion: {question}"

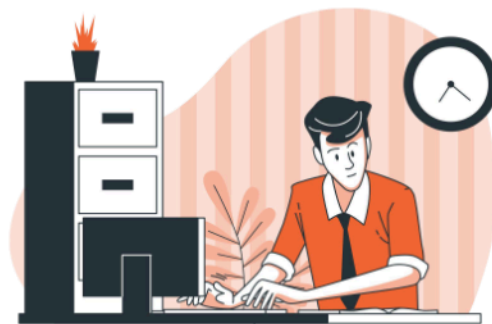
# The complete RAG chain in 4 lines of LCEL
rag_chain = (
    RunnableParallel(context=retriever, question=RunnablePassthrough())
    | prompt | ChatOpenAI(model="gpt-4o-mini") | StrOutputParser()

answer = rag_chain.invoke("What is LCEL?")
print(answer)
```

Everything comes together: retriever, passthrough, prompt, model, and parser — all connected with LCEL pipes.

Guided Hands-on

Open in Google Colab [↗](#)



Assignment



Build Your Own RAG System

Objective

Enable participants to:

- Build a complete, end-to-end RAG application using everything you learned in this course. You will create a system that can answer questions about your own documents (PDFs, text files, or web pages).

Scenario

You are an AI engineer at a company. Your team has a collection of internal documents (policies, technical guides, meeting notes). Build a RAG-powered Q&A system that lets employees ask natural language questions and get accurate, grounded answers from these documents.

[Open in Google Colab ↗](#)



Thank You



Hacktiv8id



Hacktiv8id



Hacktiv8



Hacktiv8 Indonesia